TOPICOS DE INTELIGENCIA ARTIFICIAL

Resolución del problema de las 8 reinas con Búsqueda Tabú

Alumnos:

Chiquete Martínez Karen Daniela

Samano Reyes Jordan Nayar

Grupo:

10:00 - 11:00 hrs

Docente:

Dr. Zuriel Dathan Mora Félix

Fecha:

09/10/26

1. Descripción del problema

1.1 ¿Qué se debe resolver?

Imaginemos que tenemos ocho piezas de ajedrez de tipo reina y debemos colocar una en cada columna de un tablero de 8 × 8. La reina puede desplazarse en horizontal, vertical y diagonal. Por eso, dos reinas se amenazan si comparten fila, columna o diagonal. La tarea consiste en encontrar una distribución donde no exista ningún par amenazándose [1].

Un estado es una colocación completa o parcial de las reinas que la búsqueda está considerando. En el algoritmo de este informe se usan colocaciones completas: siempre hay una reina en cada columna. Un ataque o conflicto es un par de reinas que puede capturarse mutuamente. La función objetivo se formula como minimización:

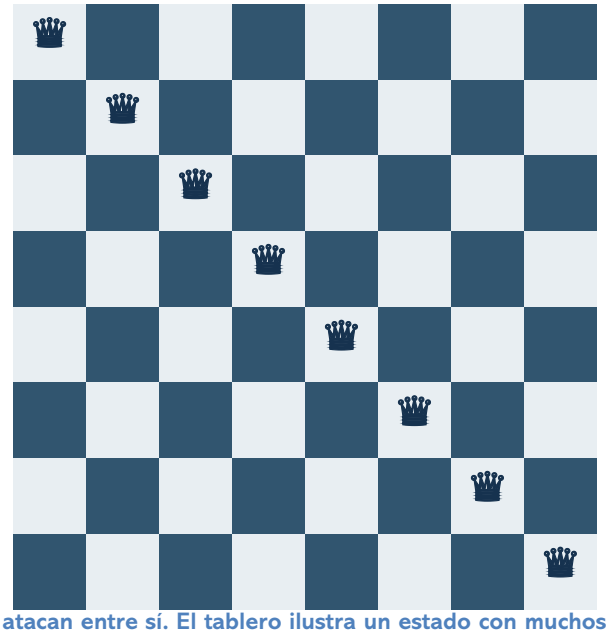
f(estado) = número de pares de reinas que se atacan. La meta es alcanzar f(estado) = 0.

Contamos pares y no solamente reinas involucradas: si tres reinas estuvieran en una misma diagonal, esa línea aportaría tres pares en conflicto. Con la representación que se presenta enseguida no pueden ocurrir ataques por fila o por columna; por eso basta con contar los ataques diagonales.

1.2 Ejemplo visual

La figura siguiente muestra un estado deliberadamente malo. Las ocho reinas están en la misma diagonal principal, así que cada pareja se amenaza. Sirve para visualizar qué significa tener conflictos; el programa real inicia desde un estado aleatorio.

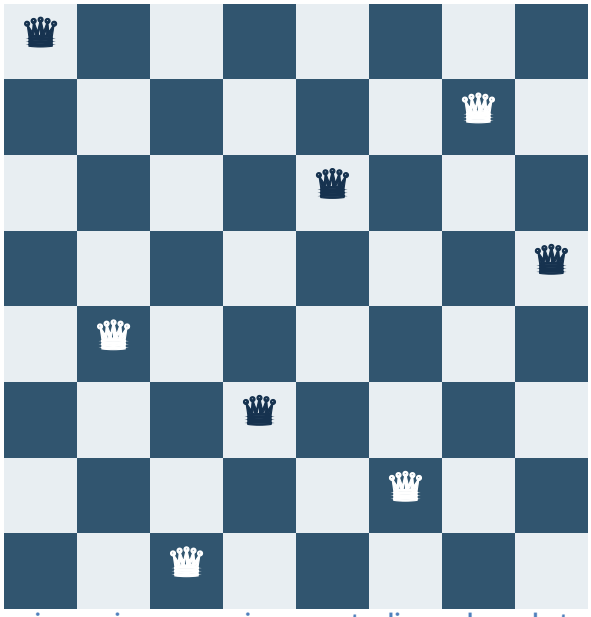
Figura 1. Ejemplo de una colocación con ataques diagonales



Las reinas de esta diagonal se atacan entre sí. El tablero ilustra un estado con muchos conflictos, no un resultado del algoritmo.

A continuación se muestra una posible solución válida. En esta distribución no hay dos reinas en una misma fila, columna o diagonal. La lista que representa las filas por columna es [0, 4, 7, 5, 2, 6, 1, 3].

Figura 2. Ejemplo de una solución sin ataques



Cada columna y fila contiene una reina, y ninguna pareja comparte diagonal; por lo tanto, la función objetivo vale cero.

2. Representación del espacio del problema P y del estado S

Para evitar confusiones, en este informe se usa P para el problema completo y E para su espacio de estados. S representa un estado candidato concreto. Esta notación es una convención explicativa: algunos libros usan otros símbolos.

2.1 Definición de P

El problema P puede describirse mediante cuatro elementos: (E, A, f, s_0). E es el conjunto de configuraciones posibles; A es el conjunto de movimientos permitidos; f es la función que cuenta los conflictos; s_0 es la configuración inicial aleatoria. Una solución es cualquier estado S* dentro de E para el que f(S*) = 0. Esta forma de definir estados, acciones y objetivo coincide con el modelado general de problemas de búsqueda y optimización [1].

2.2 Representación con una lista de ocho enteros

Se representa cada tablero con una lista de ocho posiciones. El índice representa la columna, y el valor almacenado representa la fila de la reina en esa columna. Python comienza sus índices en 0; por eso se utilizan filas y columnas de 0 a 7. Por ejemplo, [0, 4, 7, 5, 2, 6, 1, 3] significa que la reina de la columna 0 está en la fila 0, la de la columna 1 está en la fila 4, y así sucesivamente.

Elemento	Interpretación	Consecuencia
Índice de la lista	Columna de la reina	Cada índice aparece una sola vez; no hay dos reinas en una columna.
Valor en el índice	Fila de la reina	Al crear una permutación de 0 a 7, cada fila se usa una sola vez.
Intercambio de dos valores	Movimiento vecino	Cambia las filas de dos reinas y mantiene una reina por fila y columna.
Diferencia absoluta	Prueba de diagonal	Dos posiciones (c_1,r_1) y (c_2,r_2) comparten diagonal si $\|r_1-r_2\| = \|c_1-c_2\|$.

La elección de una permutación resuelve de antemano las restricciones de filas y columnas. Si permitiéramos que cada reina se moviera libremente por el tablero, tendríamos que validar las tres clases de ataque tras cada cambio. Aquí solo quedan por optimizar las diagonales, lo que simplifica tanto el código como su explicación. Este diseño también reduce el conjunto de configuraciones iniciales a 8! = 40 320 permutaciones, en lugar de 64^8 asignaciones arbitrarias con posibles repeticiones.

3. Propuesta de algoritmo

3.1 Conceptos principales

Concepto	Explicación sencilla en este ejercicio
Vecindario	Conjunto de estados que se obtienen al intercambiar las filas de dos columnas. Con ocho reinas hay C(8,2) = 28 intercambios posibles.
Movimiento	El par de columnas que se intercambia, por ejemplo (1, 6). Es una forma compacta de describir el cambio.
Lista tabú	Memoria de los últimos intercambios aceptados. Si un intercambio sigue en la lista, se evita por unas iteraciones para no volver inmediatamente sobre los pasos recientes.
Criterio de aspiración	Permite aceptar un movimiento tabú si genera una solución mejor que cualquier otra vista hasta ese momento. Así la memoria no bloquea una mejora importante.

Mejor estado global	La mejor configuración encontrada en toda la corrida; se conserva aunque el estado actual empeore temporalmente.
Criterio de paro	Se detiene al llegar a cero conflictos o al completar el máximo de iteraciones.

Un movimiento solo es tabú mientras permanece en la memoria; al salir por antigüedad vuelve a permitirse. Esta memoria adaptativa guía la exploración y reduce los ciclos [2]-[4].

3.2 Pseudocódigo

```
función BUSQUEDA_TABU(tablero_inicial, máximo, tamaño_memoria):
    actual <- copia de tablero_inicial
    mejor <- copia de actual
    memoria_prohibida <- lista vacía
    iteraciones <- 0
    aspiraciones <- 0
    bloqueos <- 0

    mientras iteraciones < máximo y costo(mejor) > 0:
        vecinos <- todos los intercambios de dos columnas
        calcular el costo de cada vecino
        mezclar vecinos y ordenarlos de menor a mayor costo

        elegir el primer vecino cuyo movimiento no sea tabú
            o cuyo costo rompa el récord global (aspiración)

        si todos están tabú y ninguno rompe el récord:
            elegir el primer vecino ordenado
            bloqueos <- bloqueos + 1

        contar si se usó aspiración
        actual <- tablero del vecino elegido
        iteraciones <- iteraciones + 1
        agregar el movimiento a memoria_prohibida
        si la memoria supera tamaño_memoria:
            quitar el movimiento más antiguo

        si costo(actual) < costo(mejor):
            mejor <- copia de actual

    devolver mejor, costo(mejor), iteraciones, aspiraciones, bloqueos
```

3.3 Secuencia para elaborar un diagrama de flujo

1. Inicio: elegir el valor de la semilla y generar una permutación aleatoria de las filas 0 a 7.
2. Calcular los conflictos iniciales. Guardar esa configuración como estado actual y mejor estado global. Inicializar vacía la lista tabú y poner los contadores en cero.

3. Comprobar el criterio de paro: si el costo es cero o se alcanzó el máximo de iteraciones, ir a la salida.
4. Generar los 28 intercambios posibles, medir sus ataques, mezclar para desempatar y ordenar por costo.
5. Descartar provisionalmente movimientos tabú, excepto cuando se active la aspiración porque el vecino mejora el mejor costo global.
6. Elegir el primer vecino ordenado que no sea tabú, o uno tabú si mejora el récord global. Si todos están tabú y ninguno mejora el récord, tomar el primer vecino ordenado y contar un bloqueo.
7. Actualizar el estado actual, sumar una iteración y un movimiento, añadir el intercambio a la memoria y retirar el más antiguo si se excedió su tamaño.
8. Actualizar el mejor estado global si la configuración actual lo supera; regresar a la comprobación del criterio de paro.
9. Salida: mostrar tablero y ataques iniciales, mejor tablero, ataques restantes, iteraciones, usos de aspiración y bloqueos, tiempo y dibujo del tablero.

4. Evaluación de herramientas y enfoques

La Búsqueda Tabú pertenece a la familia de búsqueda local. Su utilidad se entiende mejor al compararla con métodos que también cambian una configuración completa, pero deciden de otra manera qué hacer ante un paso que no mejora. No hay un método ganador para todos los problemas; la elección depende de la estructura del vecindario, del tiempo disponible y de la calidad deseada [1], [4].

Un método relacionado para problemas de restricciones es min-conflicts, que escoge una modificación intentando reducir las restricciones incumplidas. Minton y sus colaboradores estudiaron este enfoque en problemas de satisfacción de restricciones, incluido N-reinas [5]. Aquí se usa Búsqueda Tabú para hacer explícito el efecto de la memoria de movimientos.

Método	Cómo elige el siguiente estado	Fortaleza	Limitación
Escalada de colinas	Elige una mejora inmediata; en su versión más simple se detiene si ningún vecino es mejor.	Fácil de programar y puede encontrar rápidamente una solución cercana.	Puede quedarse en un mínimo local o meseta, porque no guarda memoria de lo ya intentado [1].
Búsqueda local estocástica	Selecciona al azar algún vecino o entre varios candidatos; ciertas variantes permiten pasos no óptimos.	Introduce variedad y puede escapar de una trayectoria determinista.	El resultado depende de las decisiones aleatorias y de su regla de selección; no toda variante evita ciclos.

Recocido simulado	Puede aceptar una solución peor con una probabilidad controlada por una temperatura que disminuye gradualmente [6].	La aceptación de retrocesos ayuda a salir de mínimos locales.	Requiere elegir una temperatura inicial y un plan de enfriamiento; el ajuste puede influir mucho.
Búsqueda Tabú	Escoge el mejor vecino permitido; mantiene una memoria temporal de movimientos recientes y una aspiración.	Puede continuar aunque todos los movimientos mejoren poco o empeoren, y reduce retornos inmediatos.	El tamaño y la definición de la lista tabú afectan el recorrido; añade gestión de memoria y parámetros [2]-[4].

4.1 Balance crítico

Para ocho reinas, el costo computacional es pequeño, así que la ventaja didáctica de Búsqueda Tabú es hacer visible el papel de la memoria. En un problema mayor, esa memoria puede impedir que el recorrido repita movimientos, pero también puede excluir temporalmente un movimiento útil. Una lista muy corta apenas evita ciclos; una demasiado larga puede dejar pocos vecinos admisibles y hacer el recorrido menos flexible. Por eso el tamaño tabú es un parámetro que conviene probar, no una constante universal [3], [4].

Esta versión genera y ordena todo el vecindario. Primero mezcla los vecinos para que los empates se resuelvan al azar; después recorre los ordenados y elige el primero que no sea tabú, o que cumpla aspiración. Si todos están tabú y ninguno mejora el récord global, activa el bloqueo y toma el primer vecino ordenado. Se omiten estrategias avanzadas como intensificación y memoria de largo plazo para mantener el código manejable [4].

5. Implementación en Python

El programa usa listas, funciones, ciclos, condicionales, el módulo random y el módulo time. El punto de partida es una permutación aleatoria; la semilla puede cambiarse para repetir o variar la prueba. Cada intercambio mantiene la regla de una reina por fila y columna.

```
import random
import time

# Tablero

def crear_tablero_aleatorio(n):
    """Crea un tablero con las filas en orden revuelto."""
    tablero = list(range(n))
    random.shuffle(tablero)
    return tablero


def contar_ataques(tablero):
    """Cuenta cuántas parejas de reinas están en la misma diagonal."""
    ataques = 0
```

```
    for col_a in range(len(tablero)):
        for col_b in range(col_a + 1, len(tablero)):
            # Cuántas columnas separan a las dos reinas
            distancia_horizontal = col_b - col_a
            # Cuántas filas las separan
            distancia_vertical = abs(tablero[col_a] - tablero[col_b])

            # Si se separan igual en horizontal y vertical,
            # están en una diagonal y se atacan.
            if distancia_horizontal == distancia_vertical:
                ataques += 1

    return ataques


def dibujar_tablero(tablero):
    """Muestra el tablero en pantalla: R = reina, punto = casilla vacía."""
    n = len(tablero)
    for fila in range(n):
        linea = ""
        for columna in range(n):
            if tablero[columna] == fila:
                linea += "R "
            else:
                linea += ". "
        print(linea)


# Vecinos (los tableros a los que podemos "movernos")
# Un movimiento consiste en intercambiar las filas de dos columnas.
# Con 8 columnas hay 28 intercambios posibles.

def intercambiar(tablero, col_a, col_b):
    """Devuelve una COPIA del tablero con dos columnas intercambiadas."""
    copia = tablero[:]
    guardado = copia[col_a]
    copia[col_a] = copia[col_b]
    copia[col_b] = guardado
    return copia


def generar_vecinos(tablero):
    """Genera todos los tableros que se obtienen con un intercambio.

    Cada vecino es un diccionario con tres datos:
      - "tablero":    cómo queda el tablero
      - "ataques":    cuántos ataques tiene
      - "movimiento": qué par de columnas se intercambió
    """
    vecinos = []
    n = len(tablero)
```

```
    for col_a in range(n):
        for col_b in range(col_a + 1, n):
            nuevo_tablero = intercambiar(tablero, col_a, col_b)
            vecino = {
                "tablero": nuevo_tablero,
                "ataques": contar_ataques(nuevo_tablero),
                "movimiento": (col_a, col_b),
            }
            vecinos.append(vecino)

    return vecinos


def obtener_ataques(vecino):
    """Sirve para decirle a sort() que ordene por número de ataques."""
    return vecino["ataques"]


# Regla tabú
# La "memoria prohibida" guarda los últimos movimientos hechos.
# Repetirlos está prohibido, para no dar vueltas en círculo.
# EXCEPCIÓN (aspiración): se permite un movimiento prohibido si
# logra un tablero mejor que el mejor que hemos visto en toda la búsqueda.


def elegir_vecino(vecinos, memoria_prohibida, mejor_ataques_global):
    """Escoge el mejor vecino permitido.

    Devuelve dos cosas: el vecino elegido y una palabra que explica
    por qué se eligió ("normal", "aspiracion" o "bloqueo").
    """
    # Revolvemos primero para que, si hay empates, gane uno al azar.
    # Después ordenamos de menos a más ataques (el orden relativo de
    # los empates se conserva, así que el azar se mantiene).
    random.shuffle(vecinos)
    vecinos.sort(key=obtener_ataques)

    for vecino in vecinos:
        esta_prohibido = vecino["movimiento"] in memoria_prohibida
        rompe_record = vecino["ataques"] < mejor_ataques_global

        if not esta_prohibido:
            return vecino, "normal"
        if rompe_record:
            return vecino, "aspiracion"

    # Si llegamos aquí, TODOS los movimientos estaban prohibidos
    # y ninguno rompía el récord: tomamos el mejor de todos modos.
    return vecinos[0], "bloqueo"


# Algoritmo completo
```

```
def busqueda_tabu(tablero_inicial, max_iteraciones, tamano_memoria):
    actual = tablero_inicial[:]
    mejor = actual[:]
    mejor_ataques = contar_ataques(mejor)

    memoria_prohibida = []
    iteraciones = 0
    usos_aspiracion = 0
    usos_bloqueo = 0

    # Repetimos hasta resolver el problema (0 ataques) o agotar el límite.
    while iteraciones < max_iteraciones and mejor_ataques > 0:
        vecinos = generar_vecinos(actual)
        elegido, motivo = elegir_vecino(vecinos, memoria_prohibida, mejor_ataques)

        if motivo == "aspiracion":
            usos_aspiracion += 1
        elif motivo == "bloqueo":
            usos_bloqueo += 1

        # Nos movemos al vecino elegido.
        actual = elegido["tablero"]
        iteraciones += 1

        # Anotamos el movimiento como prohibido; si la memoria se llena,
        # olvidamos el movimiento más viejo (el primero de la lista).
        memoria_prohibida.append(elegido["movimiento"])
        if len(memoria_prohibida) > tamano_memoria:
            memoria_prohibida.pop(0)

        # ¿Es el mejor tablero visto hasta ahora?
        if elegido["ataques"] < mejor_ataques:
            mejor = actual[:]
            mejor_ataques = elegido["ataques"]

    resultado = {
        "tablero": mejor,
        "ataques": mejor_ataques,
        "iteraciones": iteraciones,
        "aspiraciones": usos_aspiracion,
        "bloqueos": usos_bloqueo,
    }
    return resultado

#------------------------------------

def programa_principal():
    semilla = 35
    cantidad_reinas = 8
    max_iteraciones = 500
    tamano_memoria = 5      # Cuántos movimientos recientes se recuerdan

    random.seed(semilla)
```

```
    tablero_inicial = crear_tablero_aleatorio(cantidad_reinas)

    # Medición de lo que tarda el algoritmo.
    inicio = time.perf_counter()
    resultado = busqueda_tabu(tablero_inicial, max_iteraciones, tamano_memoria)
    fin = time.perf_counter()

    print("Tablero inicial:", tablero_inicial)
    print("Ataques al inicio:", contar_ataques(tablero_inicial))
    print()
    print("Mejor tablero encontrado:", resultado["tablero"])
    print("Ataques restantes:", resultado["ataques"])
    print("Iteraciones realizadas:", resultado["iteraciones"])
    print("Veces que se usó la aspiración:", resultado["aspiraciones"])
    print("Veces que todo estaba prohibido:", resultado["bloqueos"])
    print("Tiempo de ejecución (segundos):", round(fin - inicio, 4))
    print()
    dibujar_tablero(resultado["tablero"])


programa_principal()
```

6.1 ¿Qué hace cada parte?

- crear_tablero_aleatorio: crea la lista [0, 1, ..., n−1] y la mezcla. Al ser una permutación, cada columna y cada fila tienen exactamente una reina.
- contar_ataques: compara cada pareja de columnas. Si la distancia horizontal y la distancia vertical son iguales, las reinas comparten diagonal y se suma un ataque.
- intercambiar y generar_vecinos: cada pareja de columnas forma un movimiento. La función devuelve una copia intercambiada; el diccionario del vecino guarda el tablero, su costo y el par de columnas usado.
- elegir_vecino: mezcla la lista y luego la ordena por ataques. La mezcla aleatoria decide entre empates porque sort() conserva el orden de elementos con la misma clave. Se toma el primer movimiento normal permitido; si un movimiento tabú mejora el mejor costo global, se acepta por aspiración.
- bloqueo: si todos los vecinos están tabú y ninguno rompe el récord, la función toma el primer vecino de la lista ordenada. El contador permite saber cuántas veces se necesitó esta salida de emergencia.
- busqueda_tabu: conserva el estado actual y el mejor estado encontrado. Cada vuelta aplica un intercambio, suma una iteración y guarda el movimiento en memoria; pop(0) olvida el más antiguo cuando se rebasa el tamaño permitido.
- programa_principal: fija la semilla 35, crea el tablero de ocho reinas y mide la búsqueda con time.perf_counter(). Imprime los costos, contadores y tiempo, y dibuja el mejor tablero con R y puntos.

La copia tablero[:] evita cambiar accidentalmente la lista original al construir vecinos. El intercambio se realiza con una variable temporal llamada guardado. El programa usa únicamente listas, diccionarios sencillos, funciones, ciclos, random y time; no requiere clases ni bibliotecas de optimización.

7. Métricas, pruebas y aplicaciones prácticas

7.1 Métricas que registra el programa

El programa cuenta iteraciones y cada iteración aplica exactamente un intercambio; por eso, el número de movimientos realizados es igual al contador de iteraciones, aunque no se imprime como una línea separada. También cuenta cuántas veces se activó la aspiración y cuántas veces todos los movimientos estaban prohibidos. El tiempo se mide con time.perf_counter(), adecuado para medir intervalos cortos. El valor depende de la computadora y puede variar entre ejecuciones.

6.2 Ejemplo de prueba reproducible

La configuración incluida usa semilla 35, ocho reinas, un máximo de 500 iteraciones y memoria tabú de tamaño 5. En una ejecución de prueba, el tablero inicial fue [0, 4, 3, 7, 2, 1, 6, 5], con 5 ataques; se encontró [2, 4, 1, 7, 0, 6, 3, 5], con 0 ataques, en 3 iteraciones. La semilla permite repetir la parte aleatoria en condiciones equivalentes; al cambiarla puede cambiar el tablero inicial y el desempate. El tiempo medido depende de la computadora y no se fija como resultado esperado. Si el límite se agota con ataques restantes, se reporta ese resultado y se puede probar otra semilla o aumentar el máximo.

Prueba	Qué cambiar	Qué registrar
Repetición controlada	Mantener semilla 35 y los mismos parámetros.	Estado inicial, costo final, iteraciones, aspiraciones, bloqueos y tiempo.
Nueva corrida aleatoria	Cambiar semilla, por ejemplo a 7 o 123.	Comparar si cambian tablero, costo, contadores o tiempo.
Memoria corta	Probar tamano_memoria = 3.	Observar si visita estados similares o cuánto tarda en llegar a costo 0.
Memoria más larga	Probar tamano_memoria = 10.	Comparar recorrido y número de iteraciones; más memoria no garantiza mejor resultado.
Límite de búsqueda	Subir max_iteraciones, por ejemplo a 5000.	Comprobar si una corrida que no había resuelto el tablero consigue continuar hasta una solución.

Para una comparación justa, conviene repetir cada configuración con varias semillas y reportar el mejor resultado, el promedio y el peor resultado. Con una sola corrida no se puede afirmar que cierto tamaño tabú sea siempre superior: la inicialización aleatoria influye y este tablero es pequeño.

6.3 Guía breve para defenderlo en clase

1. Explica primero la codificación: el índice es columna y el valor es fila. La permutación garantiza una reina por columna y fila.

2. Muestra la función objetivo: cuenta parejas en diagonal; cero conflictos significa que se cumplen todas las restricciones.

3. Ejecuta el archivo una vez con semilla 35. Señala el tablero inicial, los ataques, el mejor tablero y las métricas que imprime.

4. Si el docente pide cambiar la semilla, modifica semilla = 35 dentro de programa_principal() por otro entero. La corrida será reproducible al volver a usar el mismo valor en condiciones equivalentes.

5. Si pide cambiar el tamaño de la lista tabú, modifica tamano_memoria. Un tamaño pequeño olvida movimientos pronto y uno grande los conserva durante más iteraciones; compara los resultados sin prometer que uno será mejor.

6. Si la solución no aparece en el límite, muestra el costo restante, aumenta max_iteraciones o ejecuta con otra semilla. Un método heurístico puede fallar en una corrida concreta; eso es parte de la evaluación experimental.

7. Complejidad temporal y espacial

Sea n el número de reinas, I el máximo de iteraciones ejecutadas y L el tamaño máximo de la lista tabú. Esta implementación vuelve a calcular el costo de cada vecino desde cero para que el programa sea más fácil de leer.

Parte	Costo aproximado	Razón
Conteo de conflictos	$O(n^2)$	Se revisan todos los pares de columnas.
Cantidad de vecinos	$O(n^2)$	Un intercambio por cada pareja de columnas: $n(n-1)/2$.
Una iteración completa	$O(n^4)$	$O(n^2)$ vecinos multiplicados por $O(n^2)$ para evaluar cada uno.
Corrida completa	$O(I\ n^4)$	Se repite el análisis del vecindario hasta resolver o llegar al límite.
Memoria auxiliar	$O(n^3 + L)$	Se almacenan $O(n^2)$ vecinos y cada uno contiene una copia del tablero de longitud n; la memoria tabú conserva hasta L pares de columnas.

La cota $O(I\ n^4)$ incluye la generación y evaluación de vecinos: hay $O(n^2)$ intercambios y cada tablero se copia en $O(n)$ y se evalúa en $O(n^2)$. Ordenarlos cuesta $O(n^2 \log n)$, que queda por debajo del costo de evaluación. La memoria $O(n^3 + L)$ corresponde a guardar todos los vecinos con sus copias completas; una variante que no almacenara las copias podría usar menos memoria, pero sería menos directa para explicar. Estas cotas describen esta implementación sencilla y no toda Búsqueda Tabú. Para $n = 8$ e I limitado, el trabajo es pequeño; el límite de iteraciones evita que la búsqueda siga moviéndose indefinidamente.

8. Conclusión

El problema de las 8 reinas permite practicar representación de estados, restricciones, función objetivo y búsqueda local en un ejemplo pequeño. La lista de filas por columna reduce las posibilidades de ataque que deben comprobarse y facilita generar vecinos mediante intercambios. La Búsqueda Tabú añade memoria temporal para que la búsqueda no repita enseguida movimientos recientes y conserva el mejor estado encontrado, incluso cuando se permite explorar un paso que no mejora de inmediato.

El programa es una implementación introductoria, no una demostración de que la Búsqueda Tabú siempre supere a otros métodos. Sus resultados dependen de la semilla, el vecindario, el

tamaño tabú y el límite de iteraciones. Por eso, para defenderlo en clase conviene mostrar cómo cambian las métricas en varias corridas y explicar tanto los aciertos como los casos en que una corrida no alcanza costo cero.

Referencias Bibliográficas

[1] S. Russell y P. Norvig, Artificial Intelligence: A Modern Approach, 4.ª ed. Hoboken, NJ: Pearson, 2021. ISBN 978-0-13-750513-5.

[2] F. Glover, "Tabu Search—Part I," ORSA Journal on Computing, vol. 1, núm. 3, pp. 190–206, 1989. doi: 10.1287/ijoc.1.3.190.

[3] F. Glover, "Tabu Search—Part II," ORSA Journal on Computing, vol. 2, núm. 1, pp. 4–32, 1990. doi: 10.1287/ijoc.2.1.4.

[4] F. Glover y M. Laguna, Tabu Search. Boston, MA: Kluwer Academic Publishers, 1997. doi: 10.1007/978-1-4615-6089-0.

[5] S. Minton, M. D. Johnston, A. B. Philips y P. Laird, "Minimizing conflicts: A heuristic repair method for constraint-satisfaction and scheduling problems," Artificial Intelligence, vol. 58, núms. 1–3, pp. 161–205, 1992. doi: 10.1016/0004-3702(92)90007-K.

[6] S. Kirkpatrick, C. D. Gelatt, Jr. y M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, núm. 4598, pp. 671–680, 1983. doi: 10.1126/science.220.4598.671.